

Clase 4:

Estructuras de Control : Repetición – Ciclos Inexactos

Introducción a Ciclos:

En programación existen distintas **estructuras de control algorítmicas** que permiten organizar el flujo de ejecución de un programa. Entre ellas se encuentra la **estructura de repetición**, cuya función principal es permitir que un conjunto de instrucciones se ejecute varias veces bajo determinadas condiciones.

Esta estructura está compuesta por mecanismos llamados **ciclos** (también conocidos como bucles), que hacen posible automatizar tareas repetitivas dentro de un algoritmo. Gracias a los ciclos, el programador puede evitar la duplicación innecesaria de código y diseñar soluciones más eficientes, claras y escalables.

Los ciclos se clasifican en dos grandes grupos:

- **Ciclos exactos**: cuando se conoce previamente la cantidad de repeticiones.
- **Ciclos inexactos**: cuando la cantidad de repeticiones no puede determinarse antes de la ejecución y depende de una condición lógica.

En este apunte profundizaremos en el concepto de ciclos dentro de la estructura de repetición, analizando sus características fundamentales y su funcionamiento general. Además, daremos un primer vistazo a los **ciclos inexactos**, comprendiendo su lógica, su utilidad y los contextos en los que resultan más apropiados.

¿Que son los Ciclos?

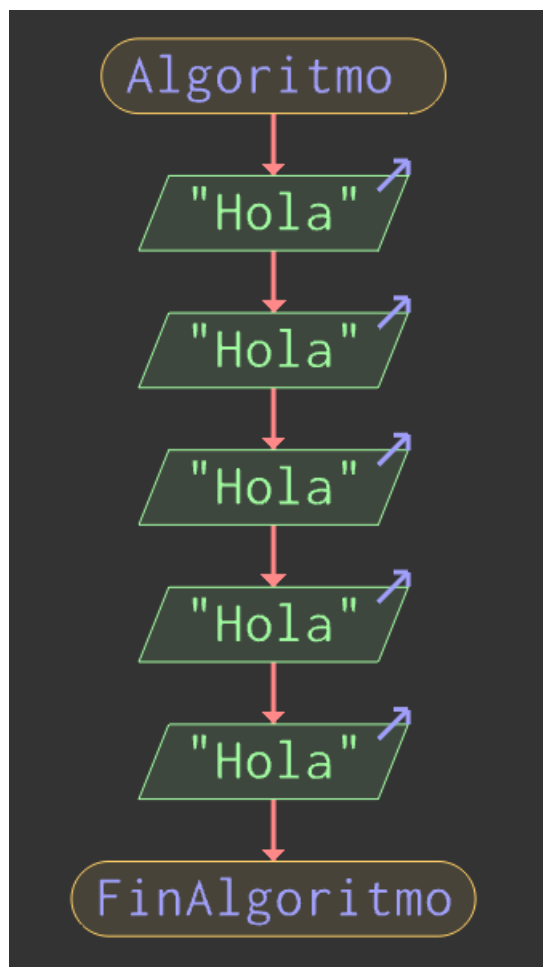
En programación, un **ciclo** (o bucle) es una estructura de control que permite ejecutar un bloque de instrucciones varias veces, mientras se cumpla una determinada condición.

Los ciclos se utilizan cuando una tarea debe repetirse:

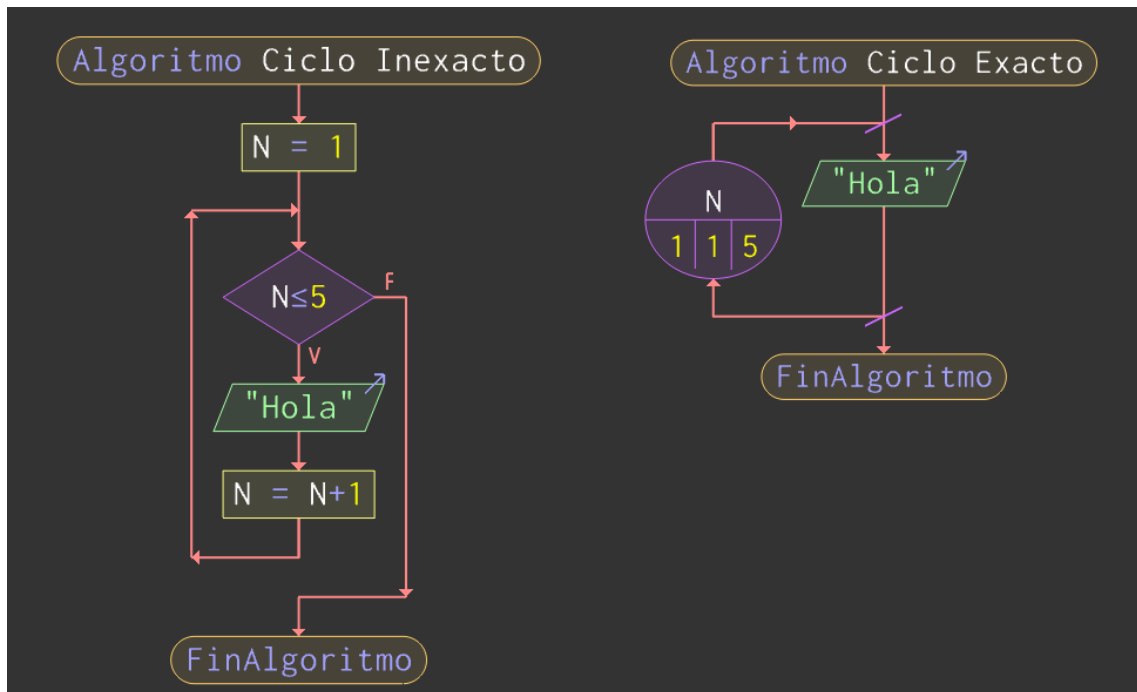
- Un número específico de veces.
- Hasta que ocurra algo.
- Mientras una condición sea verdadera.
- Hasta que el usuario ingrese un dato válido.

Sin ciclos, sería necesario escribir la misma instrucción repetidamente, lo que haría los programas largos, poco prácticos y difíciles de mantener.

Por ejemplo: Sin un ciclo, un programa que imprime “Hola” cinco veces se vería algo así:



Para evitar la repetición innecesaria de un mismo bloque de instrucciones mediante la técnica de copiar y pegar, podemos utilizar un ciclo. De esta manera, logramos que el conjunto de instrucciones se ejecute automáticamente la cantidad de veces necesaria, obteniendo una solución más eficiente, clara y mantenible. Dependiendo el tipo de ciclo que utilicemos, el resultado variaría entre algo como los siguientes:



Ciclos Inexactos

Un **ciclo inexacto** es aquel en el que no se conoce previamente cuántas veces se ejecutará el bloque de instrucciones. La repetición no está determinada por un contador fijo, sino por una **condición lógica** que se evalúa durante la ejecución del programa.

En este tipo de ciclos, el proceso continúa mientras la condición sea verdadera o hasta que ocurra un evento específico, como por ejemplo:

- Que el usuario ingrese un dato válido.
- Que se alcance un valor determinado.
- Que se detecte una señal de finalización (por ejemplo, ingresar 0).

A diferencia de los ciclos exactos (donde el número de repeticiones es conocido antes de comenzar), en los ciclos inexactos la cantidad de iteraciones depende del comportamiento del programa en tiempo de ejecución.

Este tipo de ciclo es especialmente útil en situaciones de:

- Validación de datos.
- Menús interactivos.
- Lectura de datos hasta una condición de corte.
- Procesos que dependen de eventos externos.

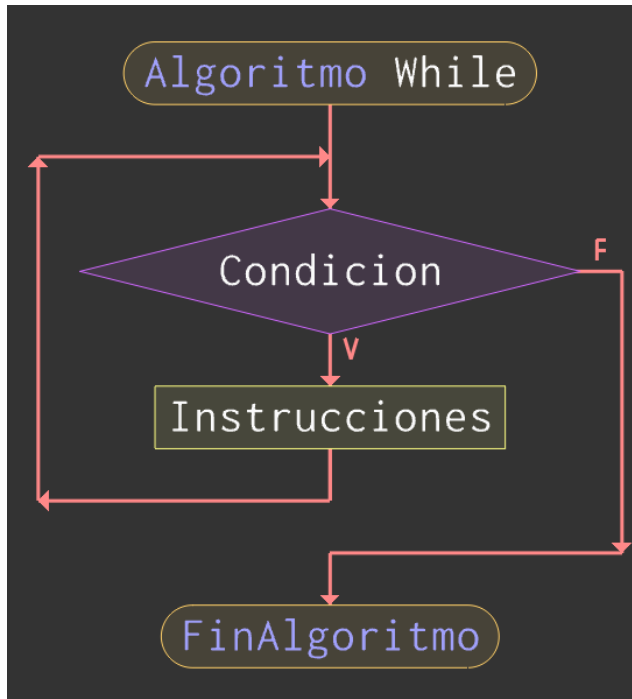
En los próximos apartados se analizarán las estructuras más comunes para implementar ciclos inexactos, como **while** y **do while**, comprendiendo su lógica de funcionamiento y sus aplicaciones prácticas dentro del diseño algorítmico.

Bucle While

El bucle **while** ejecuta un bloque de instrucciones **mientras una condición sea verdadera**.

Si la condición es falsa desde el principio, el bloque no se ejecuta ninguna vez.

Sintaxis general:



```
while(condicion){  
    instrucciones;  
}
```

Funcionamiento:

1. Se evalúa la condición.
2. Si es verdadera (true), se ejecuta el bloque.
3. Al finalizar el bloque, se vuelve a evaluar la condición.
4. Cuando la condición es falsa (false), el ciclo termina.

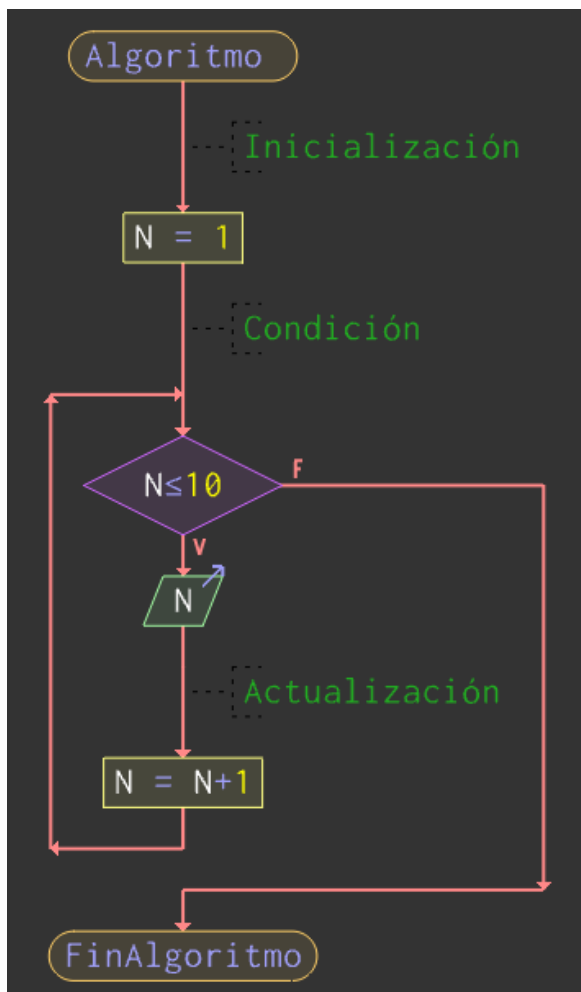
Características:

- Evalúa la condición **antes** de ejecutar.
- Puede no ejecutarse ninguna vez.
- Se usa cuando no sabemos exactamente cuántas repeticiones habrá.

Todo while correctamente construido esta conformado por tres partes:

1. Inicialización.
2. Condición.
3. Actualización.

Ejemplo: Un programa que cuente hasta 10:



1) **Inicialización:** Se define la variable de control, en este caso la variable N, la cual se le asigna el valor 1.

2) **Condición:** Se define hasta cuándo se repite el ciclo, en este caso, **hasta que N sea menor o igual a 10**.

3) **Actualización:** Una vez completadas las instrucciones a ejecutar, se le asigna un nuevo valor a la variable de control, en este caso, se incrementa el valor de N por una unidad.

Una vez completadas las 10 iteraciones, el valor final de N será 11. Dado que 11 es mayor que 10, al volver a evaluarse la condición del bucle while, el programa verificará si N es menor o igual a 10. Como esta condición ya no se cumple, el resultado será falso. En consecuencia, el ciclo finalizará y el flujo de ejecución continuará con las instrucciones siguientes dentro del programa o del diagrama de flujo.

Si quisiéramos programar este mismo algoritmo en C++, la estructura seria la siguiente:

```
#include <iostream>

using namespace std;

int main() {
    int N=1;

    while(N<=10){
        cout<<N<<endl;
        N++;
    }

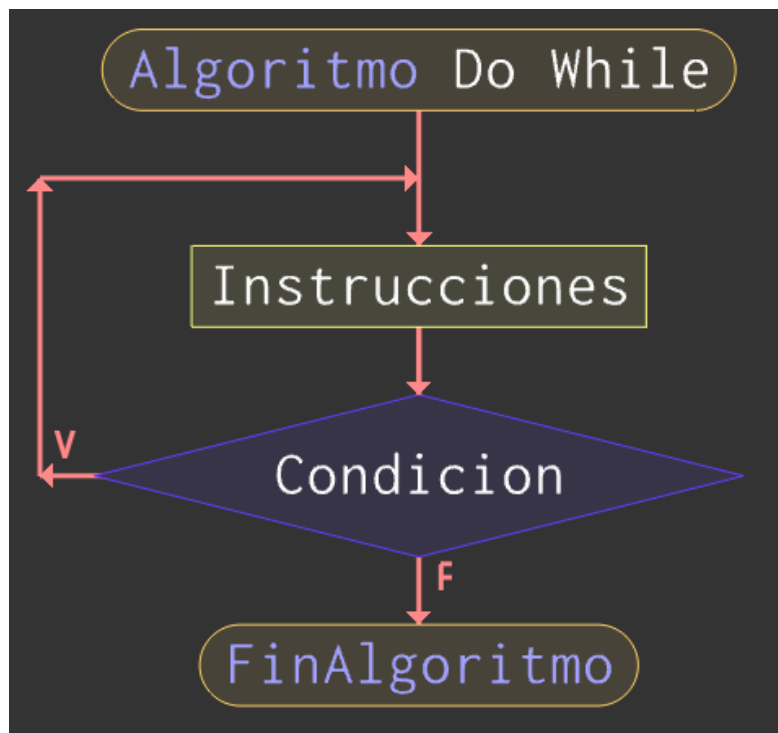
    return 0;
}
```

Bucle Do While

El **do while** es una estructura de repetición que ejecuta un bloque de instrucciones **al menos una vez**, y luego repite su ejecución mientras se cumpla una condición determinada.

A diferencia del while, la condición no se evalúa al inicio, sino **al final del ciclo**.

Sintaxis general:



```
do{  
    instrucciones;  
}while(condicion);
```

Funcionamiento:

1. Se ejecuta el bloque de instrucciones.
2. Se evalúa la condición.
3. Si la condición es verdadera, el bloque se vuelve a ejecutar.
4. Cuando la condición es falsa, el ciclo finaliza y el programa continúa con la siguiente instrucción.

Supongamos que necesitamos desarrollar un programa que solicite al usuario ingresar un número positivo. Si el número ingresado es negativo, el programa debe volver a pedir el dato hasta que el valor sea correcto.

Una posible solución utilizando while sería la siguiente:

```
#include <iostream>

using namespace std;

int main() {
    int N;

    cout << "Ingresa un positivo: "
    cin >> N;

    while (N<0) {
        cout << "Ingresa un positivo: ";
        cin >> N;
    }

    cout << "Positivo: " << N << endl;

    return 0;
}
```

Podemos observar que la instrucción para pedir el número aparece antes del while y nuevamente dentro del ciclo. Esto ocurre porque el while evalúa la condición antes de ejecutar el bloque, por lo que necesitamos una lectura inicial.

Ahora bien, utilizando do while, el mismo problema puede resolverse de forma más directa:

```
#include <iostream>

using namespace std;

int main() {
    int N;

    do{
        cout << "Ingresa un positivo: ";
        cin >> N;
    }while(N<0);

    cout << "Positivo: " << N << endl;

    return 0;
}
```

Como podemos observar, al utilizar el bucle do while evitamos tener que escribir dos veces el mismo conjunto de instrucciones. Esto se debe a que el bloque se ejecuta antes de evaluar la condición, lo que elimina la necesidad de realizar una lectura inicial fuera del ciclo. De esta manera, el código resulta más limpio, más organizado y más fácil de mantener.